IEEE Open Journal of Circuits and Systems

IEEE Xplore Digital Library
http://ieeexplore.ieee.org/Xplore

VLSI Architectures for Reed-Solomon Codes: Classic, Nested, Coupled, and Beyond

Xinmiao Zhang, *Senior Member, IEEE*

***Abstract*—Classic Reed-Solomon (RS) codes and binary Bose-Chaudhuri-Hocquenghem (BCH) codes, which can be considered as a special case of RS codes, are utilized for error correction in numerous systems, such as Flash memories, optical communications, wireless communications, magnetic storage, and deep-space probing. Additionally, RS and BCH codes are interleaved/nested to form high-gain coding schemes, including product and product-like codes and the recent generalized integrated interleaved codes, which are among the most promising candidates to address the hyper-speed and excellent-correction-capability requirements posed by next-generation terabit/s digital communications and storage. In recent developments, RS codes are also split/nested/coupled to form locally recoverable erasure codes and minimum storage regenerating codes that substantially improve the efficiency of failure recovery and enable the continued scaling of large-scale distributed storage. In this paper, prominent decoder architectures for classic RS/BCH codes are elaborated and the fundamental mathematical reformulations leading to the architectures are explained. Then the challenges and recent advancements on the decoder design of nested RS/BCH codes are highlighted. Erasure-correcting RS decoders for failure recovery are also briefly discussed. The goal of this paper is to provide comprehensive understanding of state-of-the-art VLSI architectures for classic RS/BCH codes and introduce the most recent architectures for new coding schemes built by nesting/coupling RS/BCH codes for emerging applications.**

***Index Terms*—BCH codes, Error-correcting codes, Generalized integrated interleaved codes, Key equation solver, Locally recoverable erasure codes, Reed-Solomon codes, VLSI architectures**

I. Introduction

Reed-Solomon (RS) codes [1] and binary Bose-Chaudhuri-Hocquenghem (BCH) codes [2], [3], which are a special case of RS codes in terms of en/decoding, are among the most broadly-used error-correcting codes since they have efficient en/decoders, good error-correcting capability, and wide range of supported code parameters. To name a few, they can be found in Flash memories, storage class memories, optical communications, wireless communications, digital video broadcasting, magnetic storage, and deep-space probing. Although low-density parity-check (LDPC) codes [4] are gaining more popularity over the past two decades due to their capability of efficiently utilizing the probability information from the channel to correct more errors, they require codeword length of at least thousands of bits to achieve good performance and have long decoding latency. Classic RS and BCH codes are still better candidates for systems that require short codewords, low latency, simple en/decoder, and/or have difficulties in obtaining soft channel inputs. Examples are Flash memories, storage class memories, optical transport networks, wireless sensors, and Internet-of-things (IoT) devices.

RS/BCH codes can be also used as components to build codes with higher coding gain. In product and product-like codes [5]–[8], the data symbols arranged in a two-dimensional array are interleaved row-wise and column-wise or following an alternative pattern to form RS/BCH codewords. Iterative decoding over the two dimensions is carried out to correct more errors. Decoders for product codes can achieve very high throughput since the component codes are short and the decoding of all codewords in the same dimension can be done in parallel. In particular, three-error-correcting BCH codes have been utilized as component codes to achieve throughput beyond hundreds of Gigabit/s for optical transport networks [9]–[12].

The recently-developed generalized integrated interleaved (GII) codes [13]–[15] nest a set of RS/BCH codewords to form codewords of stronger RS/BCH codes. Most of the time, the decoding only needs to be carried out on individual short sub-codewords. Besides, more errors are correctable by utilizing the nested codewords. GII codes are another category of codes that can achieve terabit/s throughput with very good error-correcting performance. Besides, they do not suffer from the error-floor problem as product codes. They are among the best candidates for systems requiring hyper throughput and very low bit error rate, such as Flash and storage class memories.

Another application of RS codes is failure recovery in distributed storage. The widely-adopted RAID-6 scheme is based on two-erasure-correcting RS codes. Distributed storage is the backbone of many new technologies, such as cloud computing and big data analytics. Besides, the continued scaling of distributed storage demands new failure recovery schemes with low latency and network bandwidth penalty. Locally recoverable erasure codes (LRC) and minimum storage regenerating (MSR) codes have been developed recently to fulfill this requirement [16]. Compared to other LRC and MSR schemes, the ones based on splitting the parities of RS codes [17], coupling RS codes [18], and modifying the nesting matrices of GII codes [15], [19] have more flexible code parameters, better trade-offs on locality and redundancy, and simpler en/decoders.

Although many VLSI architectures are available for classic RS/BCH codes, the algorithmic reformulations leading to the architectures are often described rather theoretically or not explained and their correspondence to the architectural simplifications are not well-articulated. The encoder architectures

The author is with the Department of Electrical and Computer Engineering at The Ohio State University, Columbus, OH 43210, USA. This material is based upon work supported by Kioxia Corporation and the National Science Foundation under Award No. 2011785

Digital Object Identifier: 10.1109/OJCAS.2020.3019403

2644-1225 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

are relatively simple and this paper focuses on the decoder architectures. To facilitate better understanding and utilization of existing designs, the first part of this paper elaborates the most efficient decoder architectures for conventional RS/BCH codes and highlights the fundamental mathematical modifications leading to the architectures. In particular, the key equation solver (KES) step of the decoding is more complicated than the other steps and it limits the achievable clock frequency of the decoder. The most efficient algorithms for the KES include the Berlekamp-Massey (BM) algorithm [20] and extended Euclidean algorithm [21]. The latter was initially more popular due to its inherent regularity and short critical path. Simplified architectures for this algorithm are available in [22]–[24]. However, the BM algorithm was later reformulated to eliminate the long critical path and achieve even lower complexity [25]–[29]. Hence, the focus will be given to the BM algorithm in this paper. Binary BCH codes can be considered as a special case of RS codes. The simplifications for BCH decoders are detailed after the architectures for RS decoders are introduced. Additionally, brief discussions are included for three-error-correcting BCH decoder architectures, which have very short decoding latency and are considered for storage class memories and optical communications.

The second part of this paper introduces recent developments on the GII decoder architecture design. The main bottlenecks of GII decoders lie in the KES of the nested decoding process. Unlike that for classic RS/BCH decoding, higher-order syndromes need to be incorporated into existing KES results over lower-order syndromes in the nested decoding so that redundant computations are avoided. As a result, previous KES designs for classic RS/BCH decoding are not applicable. Several algorithmic reformulations and architectures have been developed to eliminate the clock frequency bottleneck and reduce the area of the nested KES [30]–[32]. Instead of going into the details of the complicated mathematical derivations and formulas, our introduction on these architectures focuses on the bottlenecks, motivations, fundamental approaches, and remaining challenges.

The third part is dedicated to erasure-correcting RS decoders used for failure recovery. The number of failures to be recovered in many practical systems is quite limited, such as at most 3 or 4. Such erasure-correcting codes are discussed in this paper. Due to the limited correction capability, non-traditional methods can be utilized to construct (extended) RS codes and accordingly simplify the encoder and decoder [17], [33]. Besides, variations the finite field construction can be exploited to relax the constraints on the code parameters and further reduce the decoder complexity [33], [34].

The goal of this paper is twofold. The in-depth explanations of the prominent architectures for classic RS/BCH decoders in part one allow the readers to easily follow the material to develop highly-efficient RS/BCH decoders. The reviews in parts two and three cover the most recent developments on the decoder architecture design of those new codes built upon nesting/coupling RS/BCH codes for emerging technologies. They point out current state of the art and hopefully serves as the starting point for future research.

This paper is organized as follows. Section II introduces the fundamentals of RS and BCH codes. Section III and IV are dedicated to en/decoder architectures for classic RS and BCH codes, respectively. GII decoder architectures are introduced in Section V. Section VI discusses erasure-correcting RS decoders for failure recovery. Conclusions follow in Section VII.

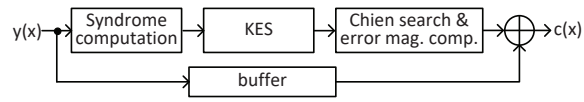

Fig. 1. Block diagram of classic syndrome-based RS decoder

II. Basics of RS and BCH codes

A t-error-correcting narrow-sense RS code over finite field $GF(2^q)$ can be constructed using a generator polynomial $g(x) = \prod_{i=1}^{2t}(x+\alpha^i)$, where α is a primitive element of $GF(2^q)$. A k-symbol message represented as a polynomial, $m(x)$, can be encoded systematically into an n-symbol codeword as $c(x) = m(x)x^{n-k} + (m(x)x^{n-k}) \mod g(x)$, where $n = k + 2t < 2^q$. RS codes are maximum distance separable (MDS) codes since $2t$ parities are the minimum amount of redundancy needed to correct t errors.

As shown in Fig. 1, classic hard-decision decoding of RS codes consists of three steps: syndrome computation, KES, and Chien search. For an (n,k) t-error-correcting RS code, let the received (corrupted) word be $y(x) = c(x) + e(x)$, where $e(x)$ is the error polynomial. Syndromes are computed as the evaluation values of $y(x)$ over the $2t$ roots of $g(x)$:

$$S_j = y(\alpha^{j+1}) = c(\alpha^{j+1}) + e(\alpha^{j+1}) = e(\alpha^{j+1}),\ (0 \le j < 2t). \tag{1}$$

If all syndromes are zero, there is no error. Otherwise, the remaining decoding steps are carried out.

Use subscript i to denote the i-th coefficient of the polynomial. Assume that $y(x)$ has $v \le t$ errors in positions $i_1, i_2, \ldots, i_v$. $X_l = \alpha^{i_l}$ are referred to as the error locators and the error magnitudes are e_{i_l} $(1 \le l \le v)$. Define the error locator polynomial as

$$\Lambda(x) = \prod_{l=1}^{v}(1 - X_l x) = \Lambda_0 + \Lambda_1 x + \cdots + \Lambda_v x^v.$$

Let $S(x) = \sum_{j=0}^{2t-1} S_j x^j$ be the syndrome polynomial. Also define the error evaluator polynomial as

$$\Omega(x) = \sum_{l=1}^{v}\left(e_{i_l} X_l \prod_{j=1, j\neq l}^{v}(1 - X_j x)\right).$$

The above polynomials satisfy the following key equation

$$\Lambda(x)S(x) = \Omega(x) \mod x^{2t}.$$

Given $S(x)$, the KES step solves this equation to get $\Lambda(x)$ and $\Omega(x)$. $\Lambda(x)$ can be computed using the iterative BM algorithm [20]. Then $\Omega(x)$ is calculated as $\Lambda(x)S(x) \mod x^{2t}$.

Algorithm A shows the pseudo codes of the BM algorithm. The superscript, '(r)', is added to denote the polynomials or values in iteration r. In this algorithm, the error locator

polynomial is initialized as $\Lambda^{(0)}(x) = 1$. In iteration r, a discrepancy coefficient, $\delta^{(r)}$, is first computed as in (2), where $L_\Lambda^{(r)}$ is the length of $\Lambda^{(r)}(x)$. Then $\Lambda^{(r)}(x)$ and its associated scratch polynomial, $B^{(r)}(x)$, are updated using $\delta^{(r)}$. After $2t$ iterations, the $\Lambda^{(2t)}(x)$ derived at the end is the error locator polynomial.

Algorithm A: the BM Algorithm
input: S_i, $(0 \le i < 2t)$
initialization: $\Lambda^{(0)}(x) = B^{(0)}(x) = 1$; $k^{(0)} = 0$;
for $r = 0$ to $2t - 1$

$$\delta^{(r)} = \sum_{i=0}^{L_\Lambda^{(r)}} \Lambda_i^{(r)} S_{r-i} \tag{2}$$

$\Lambda^{(r+1)}(x) = \Lambda^{(r)}(x) + \delta^{(r)} x B^{(r)}(x)$
if $(\delta^{(r)} \neq 0)$ and $(k^{(r)} \ge 0)$
$\quad B^{(r+1)}(x) = \Lambda^{(r)}(x)/\delta^{(r)}$; $k^{(r+1)} = -k^{(r)} - 1$
else
$\quad B^{(r+1)}(x) = xB^{(r)}(x)$; $k^{(r+1)} = k^{(r)} + 1$
output: $\Lambda(x) = \Lambda^{(2t)}(x)$;

The third decoding step computes the roots of $\Lambda(x)$ by the Chien search, which is to exhaustively evaluate $\Lambda(x)$ over each finite field element to see if the result is zero. If the number of roots of $\Lambda(x)$ equals the degree of $\Lambda(x)$, then $\Lambda(x)$ is considered a valid error locator polynomial, in which case its roots are X_l^{-1}. The evaluation values of $\Omega(x)$ can be also computed by the Chien search and the error magnitudes can be calculated using the Forney's formula [35] as

$$e_{i_l} = \Omega(X_l^{-1})/\Lambda'(X_l^{-1}), \tag{3}$$

where $(\cdot)'$ denotes the first-order derivative of the polynomial. If the degree of $\Lambda(x)$ is higher than the number of its roots, then decoding failure is declared.

A t-error-correcting binary BCH code can be constructed using a generator polynomial $g(x) = \text{LCM}(p_1(x), p_2(x), \cdots, p_{2t}(x))$, where $p_i(x)$ is the minimal polynomial [36] that has α^i as a root and LCM$(\cdot)$ denotes the least common multiple. For codes over $GF(2^q)$, the degree of $g(x)$ is around tq. Hence, an (n, k) BCH code has $n - k > 2t$ and is not MDS. The en/decoding of BCH codes follow the same algorithms for RS codes and can be simplified using the binary property. In particular, $S_{2j+1} = S_j^2$ over $GF(2^q)$ for binary codewords. Hence, the discrepancy coefficients in the odd iterations of the BM algorithm are zero and those iterations can be skipped [20]. Besides, the error evaluator polynomial and error magnitudes do not need to be computed.

III. En/decoder Architectures for Classic RS Codes

In this section, the architectures for the encoder are briefly introduced before those for the decoder of classic RS codes are discussed. For the decoder architectures, the emphasis is given to the KES step, since it is more involved than the other two and limits the achievable clock frequency of the overall decoder. The architectures that made significant advancements over prior designs are reviewed and the fundamental mathematical modifications leading to the designs are explained.

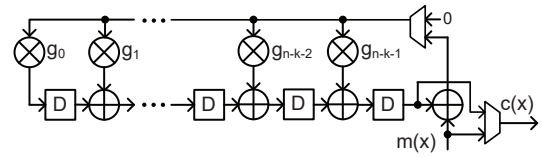

Fig. 2. Serial LFSR architecture for RS encoding

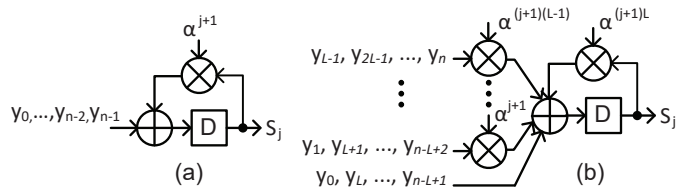

Fig. 3. Syndrome computation architectures: (a) serial; (b) L-parallel

A. Encoder architectures

The polynomial division needed for RS encoding can be implemented by a linear feedback shift register (LFSR) configured according to $g(x)$ as shown in Fig. 2. The message symbols are input serially starting from the most significant one. At the same time, they are sent to the output as the systematic part of the codeword. After the last symbol is sent in, the remainder $m(x)x^{n-k} \mod g(x)$ is located in the registers. It is shifted out to form the parity part of the codeword. Parallel RS encoders can be achieved by applying the unfolding technique [37], look-ahead register state updating [38], or parallel recursive filtering [39], [40].

B. Syndrome computation architectures for decoder

From (1), the syndrome S_j $(0 \le j < 2t)$ is the evaluation value of the received polynomial, $y(x) = y_{n-1}x^{n-1} + y_{n-2}x^{n-2} + y_{n-3}x^{n-3} + \cdots + y_0$, over α^{j+1}. Applying the Horner's rule, $y(x)$ is rewritten as

$$y(x) = (\ldots((y_{n-1}x + y_{n-2})x + y_{n-3})x + \ldots)x + y_0.$$

As a result, S_j can be computed by the simple feedback loop in Fig. 3(a). The register is initialized to zero, and the coefficients of $y(x)$ are input serially starting with the most significant one. After n clock cycles, S_j is available in the register.

A parallel syndrome computation architecture that processes L input symbols at a time is shown in Fig. 3(b). It is derived by grouping L coefficients of $y(x)$ before applying the Horner's rule. The notations in this figure assume that n is divisible by L. Otherwise, $L - (n \mod L)$ zeros should be padded before the most significant coefficient of $y(x)$. This L-parallel architecture computes a syndrome in $\lceil n/L \rceil$ clock cycles.

Parallel processing for syndrome computation is achievable in two dimensions: using multiple units to compute multiple syndromes simultaneously and/or processing multiple input symbols at a time in the computation of each syndrome as shown in Fig. 3(b). In the case that $2t$ syndromes are computed

by using $2t$ separate units simultaneously, the multipliers in the feedback loops are constant multipliers. On the other hand, sharing hardware units to compute different syndromes in a time-multiplexed manner requires general multipliers, which have much larger area and longer critical path than constant multipliers [41]. Besides, each input symbol needs to be read from the buffer multiple times, which consumes more power. Therefore, to reach higher throughput, it is more efficient to adopt $2t$ syndrome computation units and then further increase the parallelism of each unit as needed using the architecture in Fig. 3(b).

C. KES architectures for decoder

In the BM algorithm shown in Algorithm A, the $\delta^{(r)}$ computation and the following inversion and multiplication have very long data path and they lie in a feedback loop. As a result, the achievable clock frequency is low. Besides, the error evaluator polynomial is calculated as $\Omega(x) = \Lambda(x)S(x) \bmod x^{2t}$ after the error locator polynomial, $\Lambda(x)$, is derived. Although the $\Omega(x)$ computation can reuse the hardware units for calculating $\delta^{(r)}$, it requires extra clock cycles. Hence, the original BM algorithm does not lead to efficient KES implementation.

Algorithmic reformulations have been developed over the years for the BM algorithm to simplify the KES architecture. In the inversionless Berlekamp-Massey (iBM) algorithm [25], the involved polynomials are scaled to eliminate the inversion of $\delta^{(r)}$, although more multipliers are needed compared to the original BM algorithm. However, $\delta^{(r)}$ is still computed using (2), which demands a multiplier-adder tree in the data path. Besides, the polynomial updating done as linear combination after $\delta^{(r)}$ is available contributes to another multiplier and an adder in the data path. Overall, architectures based on the iBM algorithm have two multipliers and $\lceil log_2(t+1) \rceil + 1$ adders in the critical path for a t-error-correcting code. Besides, $\Omega(x)$ is still computed afterwards and extra clock cycles are required.

The reformulated (r-) iBM algorithm [26] shortens the critical path to one multiplier and one adder and eliminates the extra clock cycles for error evaluator polynomial computation. The key idea is to use a discrepancy polynomial $\hat{\Delta}(x)$, which is initialized using syndromes and is updated in parallel with $\Lambda(x)$. The discrepancy coefficient in each iteration is a coefficient of this polynomial and the polynomial updating does not need to wait for any calculation. Besides, the error magnitudes can be calculated using the coefficients of $\hat{\Delta}(x)$ and there is no need to compute a separate error evaluator polynomial. The riBM algorithm is listed below.

Algorithm B: the riBM Algorithm

input: S_i, $(0 \le i < 2t)$

initialization: $\Lambda^{(0)}(x) = B^{(0)}(x) = 1$; $k^{(0)} = 0$; $\gamma^{(0)} = 1$

$\hat{\Delta}^{(0)}(x) = \hat{\Theta}^{(0)}(x) = S(x) = \sum_{i=0}^{2t-1} S_i x^i$

for $r = 0$ to $2t-1$

1). $\Lambda^{(r+1)}(x) = \gamma^{(r)}\Lambda^{(r)}(x) + \hat{\Delta}_0^{(r)} x B^{(r)}(x)$

2). $\hat{\Delta}^{(r+1)}(x) = \gamma^{(r)}\hat{\Delta}^{(r)}(x)/x + \hat{\Delta}_0^{(r)}\hat{\Theta}^{(r)}(x)$

if $(\hat{\Delta}_0^{(r)} \neq 0)$ and $(k^{(r)} \ge 0)$

3). $B^{(r+1)}(x) = \Lambda^{(r)}(x)$; $\hat{\Theta}^{(r+1)}(x) = \hat{\Delta}^{(r)}(x)/x$

$\gamma^{(r+1)} = \hat{\Delta}_0^{(r)}$; $k^{(r+1)} = -k^{(r)} - 1$

else

4). $B^{(r+1)}(x) = xB^{(r)}(x)$; $\hat{\Theta}^{(r+1)}(x) = \hat{\Theta}^{(r)}(x)$

$\gamma^{(r+1)} = \gamma^{(r)}$; $k^{(r+1)} = k^{(r)} + 1$

output: $\Lambda(x) = \Lambda^{(2t)}(x)$; $\hat{\Delta}(x) = \hat{\Delta}^{(2t)}(x)$

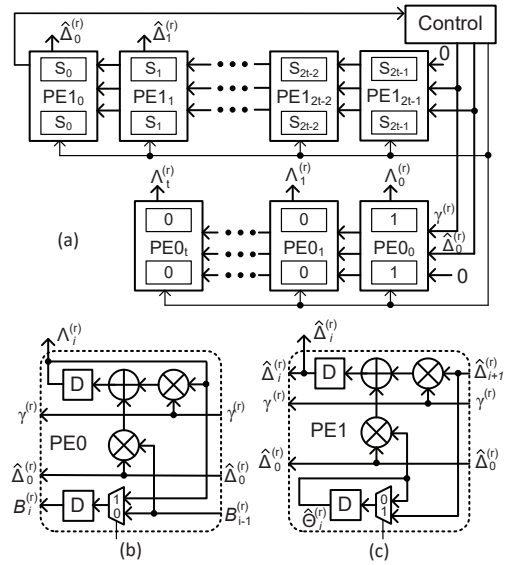

Fig. 4. riBM architecture for KES. (a) overall architecture; (b) architecture for PE0; (c) architecture for PE1

In Algorithm B, $\hat{\Theta}(x)$ is a scratch polynomial used to update $\hat{\Delta}(x)$. From (2), the discrepancy coefficient, $\delta^{(r)}$, is actually the r-th coefficient of the product $\Lambda^{(r)}(x)S(x)$. Define a discrepancy polynomial $\Delta(x) = \Lambda(x)S(x)$. It is initialized to $\Delta^{(0)}(x) = \Lambda^{(0)}(x)S(x) = S(x)$ and is updated according to $\Lambda(x)$ in parallel. Then the r-th coefficient of $\Delta^{(r)}(x)$ is $\delta^{(r)}$. To simplify the routing of the discrepancy coefficients in the circuit, $\hat{\Delta}^{(r)}(x) = \Delta^{(r)}(x)/x^r$ is used instead in the riBM algorithm, and it is initialized as $\hat{\Delta}^{(0)}(x) = \Delta^{(0)}(x)/x^0 = S(x)$. Accordingly, $\delta^{(r)}$ is simply the constant coefficient of $\hat{\Delta}^{(r)}(x)$, denoted by $\hat{\Delta}_0^{(r)}$, in every iteration. The division by x^r is achieved through iterative divisions by x over r iterations as shown in Lines 2 and 3 of Algorithm B. Since the coefficients of x^i with $i < r$ in $\Delta^{(r)}(x)$ do not contribute to later iterations, they can be eliminated over the divisions. Hence, for $\hat{\Delta}(x) = \hat{\Delta}_0 + \hat{\Delta}_1 x + \hat{\Delta}_2 x^2 + \cdots$, $\hat{\Delta}(x)/x$ is defined as $\hat{\Delta}_1 + \hat{\Delta}_2 x + \cdots$. Similarly, $\hat{\Theta}^{(r)}(x) = B^{(r)}(x)S(x)/x^r$ and it is initialized as $\hat{\Theta}^{(0)}(x) = B^{(0)}(x)S(x)/x^0 = S(x)$. For t-error-correcting RS decoding, the riBM algorithm has $2t$ iterations. At the end, $\Lambda^{(2t)}(x)$ is the error locator polynomial, whose inverse roots are the error locators, X_l, when the number of actual errors does not exceed t. Also it has been derived in [26] that the error magnitudes can be computed as

$$e_{i_l} = X_l^{-2t}\hat{\Delta}^{(2t)}(X_l^{-1})/\Lambda'^{(2t)}(X_l^{-1}).$$

The overall architecture for implementing the riBM algorithm is shown in Fig. 4(a) [26]. There are two types of processing elements (PEs). The $t+1$ PE0s are responsible for updating $\Lambda(x)$ and $B(x)$, and the $2t$ PE1s take care of

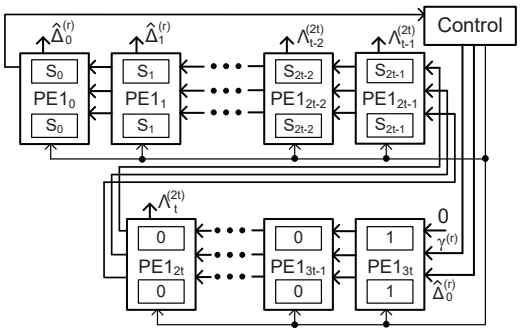

Fig. 5. RiBM architecture for KES

the updating of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$. The i-th coefficients of the polynomials are processed by the i-th PE0 and PE1. Note the different ordering of PE0s and PE1s. The architectures for PE0 and PE1 are shown in Fig. 4(b) and (c), respectively. It can be observed that the critical path consists of one multiplier and one adder. The symbols inside the PEs shown in Fig. 4(a) denote the initial values of the registers according to the polynomial initialization in Algorithm B. Over the iterations, the constant coefficient of $\hat{\Delta}^{(r)}(x)$ is always located in PE1_0. It is the discrepancy coefficient, and is fed back to the control unit and is utilized for polynomial updating.

Both $\Lambda(x)$ and $B(x)$ are initially '1'. Their degrees increase over the iterations and valid coefficients expand to the left into higher PE0s. Denote the degree of a polynomial by $\deg(\cdot)$. $\deg(B(x))$ may become higher than t. However, if the number of errors does not exceed t, $\deg(\Lambda(x)) \leq t$ and $\Lambda(x)$ is not affected by the terms of $B(x)$ whose powers of x are larger than t. Therefore, if the degree of $B(x)$ ever becomes higher than t in an iteration, its most significant coefficient (MSC) is shifted out of PE0_t and is discarded. At the end of the $2t$ iterations, $\Lambda(x) = \Lambda^{(2t)}(x)$ is located in PE0s with the least significant coefficient (LSC) in PE0_0 as shown in Fig. 4(a). On the other hand, the coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ move to the left and shrink over the iterations. Since $\deg(\Delta(x)) = \deg(\Lambda(x)S(x)) \leq (t + 2t - 1) = 3t - 1$ when the number of errors does not exceed t, $\deg(\hat{\Delta}^{(2t)}(x)) = \deg(\Delta^{(2t)}(x)/x^{2t}) < t$. Hence, at the end of the $2t$ iterations, the coefficients of $\hat{\Delta}^{(2t)}(x)$ are in PE1_0 through PE1_{t-1}.

An RiBM architecture was also proposed in [26]. The underneath algorithm is basically the same as Algorithm B. However, the updating of $\Lambda(x)$ and $B(x)$ is reorganized so that PE1s are used to implement the updating of all polynomials. It can be observed that the linear combination in Line 2 of Algorithm B has the same format as that in Line 1 divided by x. Hence, the same PE1s as used for $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ updating in Fig. 4(c) can be used to update $\Lambda(x)$, except that the coefficients of $\Lambda(x)$ are moving to the left by one PE in each iteration due to the division by x. Accordingly, the KES can be implemented by the RiBM architecture in Fig. 5 [26]. The $t + 1$ PE0s in the riBM architecture are replaced by PE1_{2t} through PE1_{3t} in the RiBM architecture. The RiBM architecture is more regular but has the same area requirement and same critical path as the riBM architecture.

The RiBM architecture has the same initialization of the polynomials as the riBM architecture. However, using PE1s, the coefficients of $\Lambda(x)$ are ordered in the same way as those of $\hat{\Delta}(x)$, *i.e*, the coefficients in the PEs from left to right correspond to increasing powers of x. Besides, the coefficients of $\Lambda(x)$ are shifted to the left by one PE in each iteration. Hence, the LSC of $\Lambda^{(r)}(x)$ is located in PE1_{3t-r}. Since $\hat{\Delta}^{(r)}(x) = S(x)\Lambda^{(r)}(x)/x^r$, $\deg(\hat{\Delta}^{(r)}(x)) = 2t - 1 + \deg(\Lambda^{(r)}(x)) - r$. When the number of errors does not exceed t, $\deg(\Lambda^{(r)}(x)) \leq t$ and hence $\deg(\hat{\Delta}^{(r)}(x)) \leq 2t - 1 + t - r < 3t - r$. Therefore, although the coefficients of $\Lambda(x)$ are shifted to the left by one PE in each iteration, they never overwrite any coefficients of $\hat{\Delta}(x)$. As a result, at the end of the $2t$ iterations, valid coefficients of $\hat{\Delta}^{(2t)}(x)$ are still located in PE1_0 through PE1_{t-1}, and those of $\Lambda^{(2t)}$ can be found in PE1_t through PE1_{2t} as shown in Fig. 5.

The complexity of the KES is further reduced from $3t + 1$ PEs as in the riBM and RiBM architectures to $2t + 1$ PEs in the enhanced Parallel Inversionless (ePI-) BM architecture [27]. The reduction is enabled by using the Horiguchi-Koetter formula to compute the error magnitudes as

$$e_{i_l} = \left(\gamma^{(2t)}\Lambda_0^{(2t)}X_l^{-(2t-1)}\right) / \left(B^{(2t)}(X_l^{-1})\Lambda'^{(2t)}(X_l^{-1})\right).$$

Instead of $\Lambda(x)$ and $\hat{\Delta}(x)$ as in the Forney's algorithm, $\Lambda(x)$ and $B(x)$ are used in this formula. In this case, $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ are only responsible for providing the discrepancy coefficients over the iterations. The constant coefficient of $\hat{\Delta}^{(r)}(x)$ is the discrepancy coefficient for iteration r. As long as this coefficient remains the same for each of the $2t$ iterations, the higher coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ do not need to be computed or stored. As a result, the higher coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ can be overwritten by the coefficients of $\Lambda(x)$ and $B(x)$ so that the total number of PEs is reduced.

Algorithm C: the ePIBM Algorithm

input: S_i, $(0 \leq i < 2t)$

initialization: $\hat{\Delta}^{(0)}(x) = S_0 + S_1x + \cdots + S_{2t-1}x^{2t-1} + x^{2t}$

$\hat{\Theta}^{(0)}(x) = S_0 + S_1x + \cdots + S_{2t-2}x^{2t-2} + x^{2t}$

$L_\Lambda^{(0)} = L_B^{(0)} = 0$

$\gamma^{(0)} = 1$; $z = 1$

for $r = 0$ to $2t - 1$

1). $\hat{\Delta}^{(r+1)}(x) = \gamma^{(r)}\hat{\Delta}^{(r)}(x)/x + \hat{\Delta}_0^{(r)}\hat{\Theta}^{(r)}(x)$

if $(\hat{\Delta}_0^{(r)} \neq 0)$ and $(L_\Lambda^{(r)} \leq L_B^{(r)})$

2). $\hat{\Theta}^{(r+1)}(x) = \hat{\Delta}^{(r)}(x)/x$

$L_\Lambda^{(r+1)} = L_B^{(r)} + 1$; $L_B^{(r+1)} = L_\Lambda^{(r)}$

$\gamma^{(r+1)} = \hat{\Delta}_0^{(r)}$; $z = \alpha^{-1}z$

else

3). if $(L_B^{(r)} = t - 1)$

4). $\hat{\Theta}^{(r+1)}(x) = \hat{\Theta}^{(r)}(x)/x$; $L_B^{(r+1)} = L_B^{(r)}$

else

5). $\hat{\Theta}^{(r+1)}(x) = \hat{\Theta}^{(r)}(x)$; $L_B^{(r+1)} = L_B^{(r)} + 1$

6). $z = \alpha^{-1}z$

$L_\Lambda^{(r+1)} = L_\Lambda^{(r)}$; $\gamma^{(r+1)} = \gamma^{(r)}$

7). $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0$

output: $\Lambda(x) = \hat{\Delta}_0^{(2t)} + \hat{\Delta}_1^{(2t)}x + \cdots + \hat{\Delta}_t^{(2t)}x^t$; z

$B(x) = \hat{\Theta}_0^{(2t)} + \hat{\Theta}_1^{(2t)}x + \cdots + \hat{\Theta}_{t-1}^{(2t)}x^{t-1}$

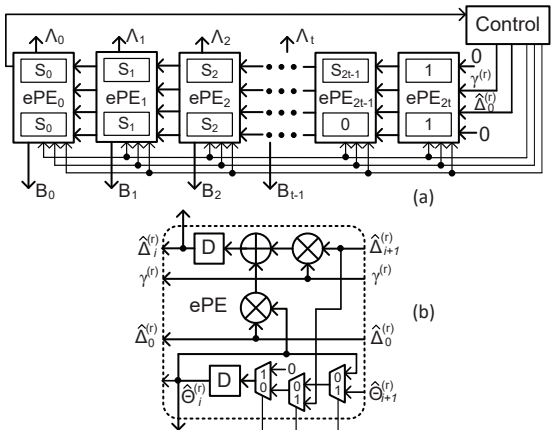

Fig. 6. ePIBM architecture for KES. (a) overall architecture; (b) ePE architecture

TABLE I
COMPLEXITIES OF KES ARCHITECTURES FOR t-ERROR-CORRECTING RS DECODING

	PEs	adders	multipliers	muxes	registers	clocks	crit. path.
riBM	$3t+1$	$3t+1$	$6t+2$	$3t+1$	$6t+2$	$2t$	1mult+1add
RiBM	$3t+1$	$3t+1$	$6t+2$	$3t+1$	$6t+2$	$2t$	1mult+1add
ePIBM	$2t+1$	$2t+1$	$4t+2$	$6t+3$	$4t+2$	$2t$	1mult+1add

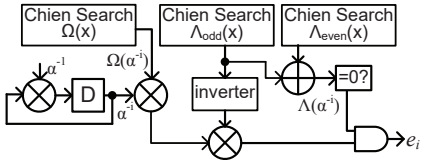

Fig. 7. Block diagram for error locator and magnitude computation using Forney's formula

The ePIBM algorithm is listed in Algorithm C and the corresponding implementation architecture is shown in Fig. 6 [27]. To better associate the architecture with the polynomial updating, $\Lambda(x)$ and $B(x)$ are integrated into the $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ polynomials, respectively, in Algorithm C. The x^{2t} terms in the $\hat{\Delta}^{(0)}(x)$ and $\hat{\Theta}^{(0)}(x)$ initialization actually represent $\Lambda^{(0)}(x) = 1$ and $B^{(0)}(x) = 1$, respectively. The RiBM algorithm in [26] uses similar notations. In Algorithm C, $L_\Lambda^{(r)}$ and $L_B^{(r)}$ track the length of $\Lambda^{(r)}(x)$ and $B^{(r)}(x)$, respectively. The $k^{(r)}$ in Algorithm B equals $L_B^{(r)} - L_\Lambda^{(r)}$. The elimination of the higher coefficients of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ is done through setting $\hat{\Theta}_{2t-r-2}^{(r+1)}$ to zero as in Line 7 of Algorithm C. $\hat{\Theta}_{2t-1}^{(0)} = 0$ is also reflected in the $\hat{\Theta}^{(0)}(x)$ initialization. By making this modification, it has been proved in [27] that $\hat{\Delta}_0^{(r)}, \hat{\Delta}_1^{(r)}, \cdots, \hat{\Delta}_{2t-1-r}^{(r)}$ and $\hat{\Theta}_0^{(r)}, \hat{\Theta}_1^{(r)}, \cdots, \hat{\Theta}_{2t-2-r}^{(r)}$ for $0 \le r < 2t$ are intact, while the higher coefficients become zero. Accordingly, the discrepancy coefficients $\hat{\Delta}_0^{(r)}$ remain the same for all $2t$ iterations. On the other hand, since the PE holding the MSCs of $\hat{\Delta}(x)$ and $\hat{\Theta}(x)$ becomes vacant in the next iteration, it can be used to update and hold the expanding coefficients of $\Lambda(x)$ and $B(x)$ shifting to the left. As a result, the initial $\Lambda^{(0)}(x) = B^{(0)}(x) = 1$ can be loaded into PE_{2t} right next to the PEs holding the initial $\hat{\Delta}^{(0)}(x)$ and $\hat{\Theta}^{(0)}(x)$ as shown in Fig. 6(a). Accordingly, the coefficients of $\Lambda^{(r)}(x)$ and $B^{(r)}(x)$ are seamlessly integrated with those of $\hat{\Delta}^{(r)}(x)$ and $\hat{\Theta}^{(r)}(x)$ and the total number of PEs is reduced to $2t+1$.

The length of $B(x)$ may reach $2t-1$ in the worst case instead of $t-1$ as the maximum degree of $\hat{\Delta}(x)$. If $B(x)$ is used to compute the error magnitudes, the complexity of the following Chien search becomes much higher. To address this issue, it was found in [27] that if the number of errors does not exceed t and $L_B^{(K)}$ reaches $t-1$ in an iteration $r = K$, then the inverse roots of $\Lambda^{(K+1)}(x)$ are already the error locators. As a result, the error magnitudes can be computed as

$$e_{i_l} = \left(\gamma^{(K)}\Lambda_0^{(K+1)}X_l^{-(K-1)}\right) / \left(B^{(K)}(X_l^{-1})\Lambda'^{(K+1)}(X_l^{-1})\right). \tag{4}$$

The lengths of the polynomials involved in the above formula are the same as those of the polynomials in the Forney's algorithm. Accordingly, the following Chien search step has the same complexity.

The coefficients of $\Lambda(x)$ and $B(x)$ are shifting in the ePIBM architecture over the iterations. If the KES stops at iteration $r = K$, extra multiplexers are needed to read the coefficients of $B^{(K)}(x)$ and $\Lambda^{(K+1)}(x)$. Instead, the ePIBM algorithm continues for $2t$ iterations. In each remaining iteration $r > K$, $B^{(K)}(x)$ is shifted to the left by one PE as done by Lines 3 and 4 of Algorithm C. Hence, at the end of the last iteration, $B^{(K)}(x)$ is located at the leftmost t PEs. Besides, the discrepancy coefficient is zero and $\Lambda(x)$ is simply scaled in each remaining iteration. The scaling of $\Lambda'(X_l^{-1})$ in (4) is canceled out by the scaling of Λ_0. Therefore, there is no need to modify the updating of $\Lambda(x)$, and its coefficients located in the leftmost $t+1$ PEs at the end of the $2t$ iterations can be used directly in (4) to compute the error magnitudes. $z = \alpha^{-(K-1)}$ at the end of Algorithm C. It is used to iteratively compute $X_l^{-(K-1)}$ with $X_l = \alpha, \alpha^2, \alpha^3, \cdots$ in synchronous with the Chien search on $B(x)$ and $\Lambda(x)$ for error magnitude computation as will be explained in the next subsection. Fig. 6(b) shows the details of the enhanced (e-) PE unit used in the ePIBM architecture. It is very similar to the PE1 unit for the riBM and RIBM architectures, except that more multiplexers are included to set $\hat{\Theta}_{2t-r-2}^{(r+1)} = 0$ and shift $B(x)$. Its critical path also has one multiplier and one adder.

The complexities of the aforementioned fast KES architectures for t-error-correcting RS decoding are summarized in Table I. Since the complexities of various PEs are different, the total numbers of adders, multipliers, etc. are also listed. If more clock cycles can be spent on the KES, the folding technique can be applied to reduce the area requirement [37], [42]. In an N-folded architecture, the KES takes $2tN$ clock cycles and the number of logic units is reduced by a factor of N, although the number of registers is unchanged.

D. Chien search architectures for decoder

The roots of a polynomial over finite field are found by exhaustive search, which can be carried out efficiently by the Chien search. To avoid inverting the roots, the Chien search for $\Lambda(x)$ can be done in the order of α^{-i} for $i = 0, 1, \cdots, n-1$. If $\Lambda(\alpha^{-i}) = 0$, then the i-th symbol is

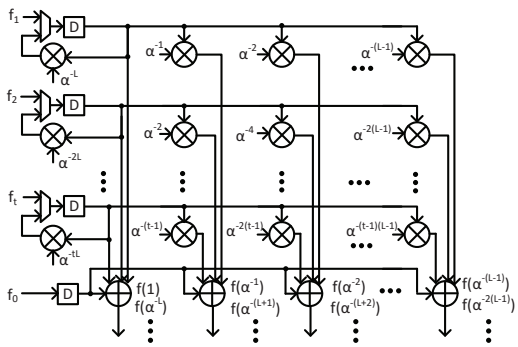

Fig. 8. L-parallel Chien search architecture

erroneous. The evaluation values involved in the Forney's and Horiguchi-Koetter error magnitude computation formulas can be also calculated by the Chien search. Besides, for $\Lambda(x) = \Lambda_0 + \Lambda_1 x + \Lambda_2 x^2 + \Lambda_3 x^3 \cdots$, $\Lambda'(x) = \Lambda_1 + \Lambda_3 x^2 + \cdots$ over $GF(2^q)$. Hence, $X_l^{-1}\Lambda'(X_l^{-1}) = \Lambda_{odd}(X_l^{-1})$, where $\Lambda_{odd}(x)$ consists of the odd terms of $\Lambda(x)$. Therefore, the Chien search for $\Lambda(x)$ is split into even and odd parts, and the odd part is shared for error magnitude computation. Take the Forney's formula in (3) as an example. It is modified as $e_{i_l} = X_l^{-1}\Omega(X_l^{-1})/\Lambda_{odd}(X_l^{-1})$. Accordingly, the error locators and magnitudes can be calculated in parallel according to the block diagram in Fig. 7. The feedback loop in the bottom left is to iteratively derive $\alpha^0, \alpha^{-1}, \alpha^{-2}, \cdots$ as X_l^{-1}. The $X_l^{-(K-1)}$ in the Horiguchi-Koetter formula in (4) can be generated by a similar feedback loop utilizing the z value computed from Algorithm C. In the following, we focus on the Chien search architecture for a general polynomial $f(x)$ of degree t.

A Chien search architecture that searches over L finite field elements in each clock cycle for a polynomial of degree t is shown in Fig. 8. It consists of an array of $t \times L$ constant multipliers. In the Chien search architectures for $\Lambda_{even}(x)$ and $\Lambda_{odd}(x)$, half of the rows are eliminated since half of the polynomial coefficients are zero. In the first clock cycle, the polynomial coefficients are loaded into the registers. The L outputs are the L evaluation values over α^0 through α^{L-1}. In each of the second and later clock cycles, the multiplexers pass the multiplier outputs from the feedback loops, and the evaluation values over the next L field elements are calculated. For a code of length n, this architecture finishes the search in $\lceil n/L \rceil$ clock cycles.

Multiplication over $GF(2^q)$ is defined as multiplying the polynomial representations of the two operands and then modulo reducing the product by the degree-q irreducible polynomial used for field construction. Details about the implementation of finite field operations are available in [41]. In the case that one operand is constant, the polynomial multiplication and modulo reduction can be combined as the multiplication of a $q \times q$ constant binary matrix. Using this constant multiplier implementation, the area of the Chien search architecture can be reduced by applying substructure sharing within each multiplier, among the multipliers in the same row [43], and when the multiplier outputs are added up [44]. Besides, using the polynomial multiplication approach, the modulo reductions can be put off until after the products for the multipliers in the same column are added up [45]. However, more bits need to be added up and this approach leads to higher gate count when t is not small. Additionally, an efficient Chien search architecture that can be re-configured to handle polynomials of higher degree in a time-multiplexed manner was proposed in [30].

Various schemes have also been developed to reduce the power consumption of the Chien search. At the cost of L additional registers and twice the clock cycles, the Chien search is divided into two steps [46]. The higher $q/2$ bits of the evaluation values are only computed when the lower $q/2$ bits are zero. Another approach is to factor $(x + \alpha)$ out of $f(x)$ and turn off the hardware units for higher coefficients right after α is found to be a root [47]. The serial design in [47] was also extended for parallel Chien search in [48].

IV. En/decoder architectures for classic binary BCH codes

BCH codes share the same en/decoding steps as RS codes. Since the generator polynomial is binary, the multipliers in the encoder architecture in Fig. 2 are replaced by wires or no connections. Besides, utilizing the binary property, further simplifications can be made on parallel BCH encoders derived by unfolding [49]–[51] and state updating [52]–[56]. For BCH decoding, the syndrome computation architectures in Fig. 3 are easily simplified. The Chien search over the error evaluator polynomial and the units for error magnitude computation in Fig. 7 are eliminated. However, non-trivial simplifications can be further made on the KES. Moreover, three-error-correcting BCH decoding achieves very short latency by computing the error locator polynomial and its roots without using iterative KES or Chien search. This section first presents simplified KES architectures for binary BCH decoding. Then three-error-correcting BCH decoder architectures are briefly discussed.

A. KES architectures for binary BCH decoding

For binary BCH codes, the discrepancy coefficients in the odd iterations of the original BM, riBM, RiBM, and ePIBM algorithms are zero since $S_{2j+1} = S_j^2$ over $GF(2^q)$. Hence, the computations for an odd iteration $r+1$ can be easily combined into even iteration r, and the odd iterations can be skipped. Take the ePIBM algorithm in Algorithm C as an example. In iteration $r+1$, since the discrepancy coefficient $\hat{\Delta}_0^{(r+1)}$ is zero, $\hat{\Delta}^{(r+2)}(x)$ should be updated as $\gamma^{(r+1)}\hat{\Delta}^{(r+1)}(x)/x$. However, the scaling by $\gamma^{(r+1)}$ can be skipped without affecting the decoding results. Hence, combining the computations in iterations r and $r+1$, the PIBM algorithm [29] listed below for the KES in BCH decoding is developed.

Algorithm D: PIBM Algorithm for BCH KES

input: S_i, $(0 \le i < 2t)$

initialization: $\hat{\Delta}^{(0)}(x) = S_0 + S_2 x^2 + \cdots + S_{2t-2}x^{2t-2} + x^{2t}$

$\hat{\Theta}^{(0)}(x) = S_1 + S_3 x^2 + \cdots + S_{2t-3}x^{2t-4} + x^{2t-1}$

$L_\Lambda^{(0)} = 0$; $L_B^{(0)} = -1$; $\gamma^{(0)} = 1$

for $r = 0, 2, 4, \cdots, 2(t-1)$

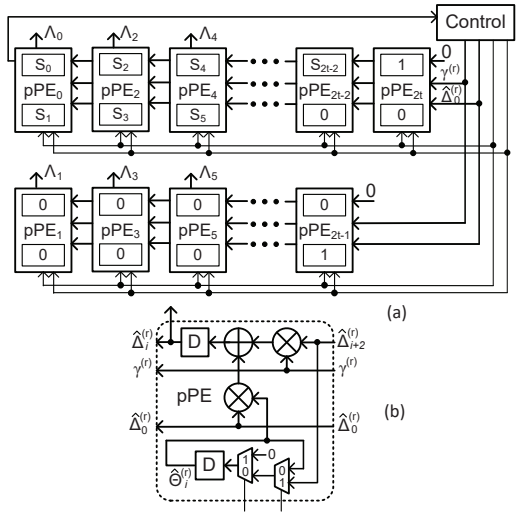

Fig. 9. PIBM architecture for binary BCH KES. (a) overall architecture; (b) pPE architecture

1). $\hat{\Delta}^{(r+2)}(x) = \gamma^{(r)}\hat{\Delta}^{(r)}(x)/x^2 + \hat{\Delta}_0^{(r)}\hat{\Theta}^{(r)}(x)$
if ($\hat{\Delta}_0^{(r)} \neq 0$) and ($L_\Lambda^{(r)} \leq L_B^{(r)} + 1$)
2). $\hat{\Theta}^{(r+2)}(x) = \hat{\Delta}^{(r)}(x)/x^2$
$L_\Lambda^{(r+2)} = L_B^{(r)} + 2$; $L_B^{(r+2)} = L_\Lambda^{(r)}$
$\gamma^{(r+2)} = \hat{\Delta}_0^{(r)}$
else
3). $\hat{\Theta}^{(r+2)}(x) = \hat{\Theta}^{(r)}(x)$
4). $L_\Lambda^{(r+2)} = L_\Lambda^{(r)}$; $L_B^{(r+2)} = L_B^{(r)} + 2$
$\gamma^{(r+2)} = \gamma^{(r)}$
5). $\hat{\Theta}_{2t-r-4}^{(r+1)} = 0$
output: $\Lambda(x) = \hat{\Delta}_0^{(2t)} + \hat{\Delta}_1^{(2t)} + \cdots + \hat{\Delta}_t^{(2t)}$

Besides combining the iterations, another three modifications have been made in the PIBM algorithm from the ePIBM algorithm: i) since $B(x)$ is not needed for error magnitude computation in BCH decoding, the enhancement in Lines 3-4 of Algorithm C for keeping $B(x)$ unchanged is not needed; ii) to facilitate the separation of the terms with even and odd powers of x, the initialization of $\hat{\Theta}(x)$ and accordingly the related updating are modified so that only divisions by x^2 are involved; iii) since the odd discrepancy coefficients do not need to be computed, the coefficients with odd powers of x in the initial polynomials are set to zero, which helps to reduce the power consumption. The simplified inversionless (Si-) BM algorithm [28] is equivalent to the PIBM algorithm, except that the odd terms of the polynomials are not initialized to zero.

The PIBM architecture is shown in Fig. 9. The $2t+1$ pPEs are identical. The $t+1$ pPEs in the top row handle the even coefficients and those in the bottom update the odd coefficients. Hence, the division of $\hat{\Delta}(x)$ by x^2 is realized by shifting the coefficients to the adjacent PEs in the same row. To synchronize with the modified $\hat{\Theta}(x)$ initialization, $B(x)$ is initialized to x^{-1}. It is represented as the x^{2t-1} term in $\hat{\Theta}^{(0)}(x)$ in Algorithm D and corresponds to the initial '1' in the register of pPE$_{2t-1}$ in the bottom row of Fig. 9(a). Similarly, $\Lambda^{(0)}(x) = 1$ is actually the x^{2t} term in $\hat{\Delta}^{(0)}(x)$ and corresponds to the initial '1' in the register of pPE$_{2t}$. After t iterations, the even and odd coefficients of $\Lambda(x)$ are located in the $\lceil (t+1)/2 \rceil$ and $\lfloor (t+1)/2 \rfloor$ pPEs in the top and bottom rows, respectively. The pPE architecture is shown in Fig. 9(b). It is essentially the same as the ePE unit in Fig. 6(b) for the ePIBM architecture, except that the multiplexer in the bottom right corner for keeping $B(x)$ unchanged is eliminated.

In the PIBM architecture, the even and odd coefficients are separated. The odd coefficients do not contribute to the computations of the even coefficients or the discrepancy coefficients. The PEs in the bottom row are only responsible for generating the odd coefficients of $\Lambda(x)$. Using the pPE architecture in Fig. 9(b), the odd coefficients are shifted to the left by one PE in each clock cycle, although the number of odd coefficients in $\Lambda(x)$ does not exceed $\lfloor (t+1)/2 \rfloor$. Instead of using the left-shifting pPEs, PE0s as in Fig. 4(b) can be adopted to update the odd coefficients of $\Lambda(x)$ in place in the same way as in the riBM architecture. In this case, only $h = \lfloor (t+1)/2 \rfloor$ PE0s instead of t pPEs are needed to handle the odd coefficients of $\Lambda(x)$, and the total number of PEs is reduced to around $1.5t+1$ [29]. The resulted architecture is shown in Fig. 10 and it is referred to as the revised (r-)PIBM architecture.

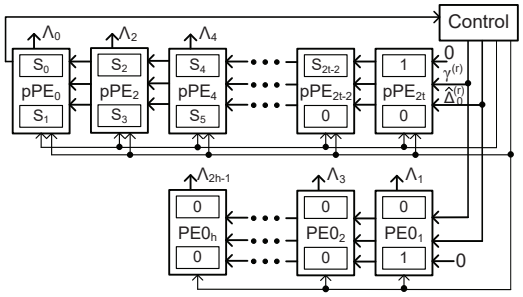

Fig. 10. rPIBM architecture for binary BCH KES

TABLE II
COMPLEXITIES OF KES ARCHITECTURES FOR t-ERROR-CORRECTING BCH DECODING

	PEs	adders	multipliers	muxes	registers	clocks	crit. path.
PIBM	$2t+1$	$2t+1$	$4t+2$	$4t+2$	$4t+2$	t	1mult+1add
rPiBM	$1.5t+1$	$1.5t+1$	$3t+2$	$2.5t+2$	$3t+2$	t	1mult+1add

The complexities of the above KES architectures for binary BCH decoding are summarized in Table II. To simplify the notations, t is assumed to be even. Similarly, if more clock cycles can be spent on the KES, the folding technique can be applied to reduce the area.

B. Three-error-correcting BCH decoder

Three-error-correcting BCH codes have very short decoding latency. They can be also utilized as the component codes of product or product-like codes to achieve very high decoding speed and good error-correcting performance [9]–[12].

The constant coefficient of the error locator polynomial, Λ_0, is always '1' from the definition. When $t = 3$, the other coefficients can be computed using the Peterson's algorithm [36] as $\Lambda_1 = S_0, \Lambda_2 = (S_0^2 S_2 + S_4)/(S_0^3 + S_2)$, and $\Lambda_3 = (S_0^3 + S_2) + S_0\Lambda_2$. Compared to the BM algorithm, these computations require smaller area and are implementable

by combinational logic. Nevertheless, the data path should be pipelined to increase the clock frequency, in which case the latency of these computations is a few clock cycles.

The roots of a degree-3 polynomial over finite field can be computed directly without the Chien search. The roots of $\Lambda^*(x) = x^3 + \Lambda_1 x^2 + \Lambda_2 x + \Lambda_3$ are the error locators. Let $a = \Lambda_1^2 + \Lambda_2$, $b = \Lambda_3 + \Lambda_1\Lambda_2$, and $c = ba^{-3/2}$. By variable substitutions $y = x + \Lambda_1$ and $z = ya^{-1/2}$, $\Lambda^*(x)$ is converted to the format of $z^3 + z + c$ [57]. Then the roots can be found by looking up pre-computed roots of $z^3 + z + c$ for every $c \in GF(2^q)$. The size of such a look-up table (LUT) is $3q2^q$ and is big for long codes. Utilizing normal basis of finite fields, the LUT is compressed to a fraction of its original size in [10]. With small overhead in the LUT address generation, only one root for each c starting with a certain pre-fix is stored, and the other two are computed using simple logic. The size of the LUT is minimized in [11]. A valid degree-3 error locator polynomial should have three distinct roots. Not every $c \in GF(2^q)$ satisfies this requirement and all the valid c can be divided into conjugacy classes. Accordingly, only one root is stored for each conjugacy class of valid c.

V. VLSI ARCHITECTURES FOR GII CODES

A codeword of a GII $[m, v]$ code [13], [14] is divided into m sub-codewords, $c_0, c_1, \cdots, c_{m-1}$, each of which belongs to a RS/BCH code $\mathcal{C}_0$ that is t_0-error-correcting. Besides, the nested codeword defined as $\tilde{c}_l = \sum_{i=0}^{m-1} \alpha^{il} c_i$ $(0 \le l < v)$ belongs to a stronger RS/BCH code $\mathcal{C}_{v-l}$, which is capable of correcting t_{v-l} errors. Without loss of generality, $t_0 < t_1 \le t_2 \le \cdots \le t_v$. Formally, the GII code is defined as

$$\mathcal{C} \triangleq \left\{ c = [c_0, \cdots, c_{m-1}] : c_i \in \mathcal{C}_0, \tilde{c}_l = \sum_{i=0}^{m-1} \alpha^{il} c_i \in \mathcal{C}_{v-l}, 0 \le l < v \right\}. \tag{5}$$

The nesting scheme can be also represented by a matrix G with $G_{l,i} = \alpha^{il}$ $(0 \le l < v, 0 \le i < m)$. Sort the number of errors in the sub-codewords as $\tau_0 \le \tau_1 \le \cdots \le \tau_{m-1}$. The errors are correctable if $\tau_{m-v-1} \le t_0$ and $\tau_{m-v-1+i} \le t_i$ for $1 \le i \le v$ [13], [14]. The above two-layer GII codes have been generalized to three-layer GII codes [15], which apply nestings also to groups of sub-codewords besides across all sub-codewords. GII codes have two advantages: i) most of time, the errors are corrected by decoding the shorter individual sub-codewords and the decoder can achieve very high throughput with low complexity; ii) more errors are correctable by utilizing the nested codes to reach the reliability goal with lower redundancy. Due to these reasons, GII codes are among the best candidates for error correction in next-generation terabit/s systems.

A systematic encoding approach for two-layer GII codes is available in [14] and it has been modified for three-layer codes in [58]. For BCH-based GII codes, the encoder can be simplified by modifying the nesting scheme [59]. GII encoders basically consist of LFSRs with the inputs and outputs multiplied by matrices pre-computed according to the nesting scheme. On the other hand, GII decoder design is much more challenging and is the focus of this section.

GII decoding consists of two stages. The first stage is traditional RS/BCH decoding over individual sub-codewords and the architectures presented in previous sections can be used. If any sub-codeword has more than t_0 errors, the second-stage nested decoding is activated. $2t$ syndromes are needed to correct t errors. Although only $2t_0$ syndromes can be computed from each sub-codeword using (1), more syndromes can be derived from the nested codewords since they have higher correcting capability. Let $y_i(x)$ be the i-th received sub-codeword. Then the syndromes for the nested codewords are $\tilde{S}_j^{(l)} = \sum_{i=0}^{m-1} \alpha^{il} y_i(\alpha^{j+1})$ $(0 \le l < v, 0 \le j < 2t_{v-l})$. These syndromes are connected to the syndromes of the sub-codewords by the same nesting used for code construction, such as that in (5). Hence, assuming that the indices of the sub-codewords with more than t_0 errors are $i_0, i_1, \cdots, i_{b-1}$ $(b \le v)$, the $2t_0 \le j < 2t_1$ higher-order syndromes for each of these sub-codewords can be computed by

$$\left[S_j^{(i_0)}, S_j^{(i_1)}, \cdots, S_j^{(i_{b-1})}\right]' = A^{-1} \left[\tilde{S}_j^{(0)}, \tilde{S}_j^{(1)}, \cdots, \tilde{S}_j^{(b-1)}\right]'. \tag{6}$$

Here A is a submatrix formed by the first b rows and columns $i_0, i_1, \cdots, i_{b-1}$ of the nesting matrix G. Using $2t_1$ syndromes, the sub-codewords with up to t_1 errors are corrected. Then the $2t_1 \le j < 2t_2$ higher-order syndromes are calculated for each of the remaining sub-codewords using an equation similar to (6) and the sub-codewords with up to t_2 errors are corrected. This nested decoding is repeated for up to $b \le v$ rounds.

In each nested decoding round, once the higher-order syndromes are derived, the same KES and Chien search as in the traditional RS/BCH decoding can be carried out to correct the errors. However, the KES for taking care of $2t_i$ syndromes has $2t_i$ iterations for RS codes and re-starting the KES in each nested decoding round causes long latency. The original BM [20] and iBM [25] algorithms incorporate one more syndrome in each iteration. Using these algorithms, the KES for a nested decoding round can start with the results from the sub-codeword decoding or previous nested decoding round [14]. In this case, only $2(t_i - t_{i-1})$ iterations are needed in the KES for the i-th nested decoding round. Nevertheless, the BM and iBM algorithms have long critical path as explained previously. On the other hand, the riBM and other algorithms with short critical path have one thing in common: all syndromes are needed in the very beginning for polynomial initialization. Hence they can not be utilized to incorporate the higher-order syndromes in the nested decoding. It was proposed in [60] to re-initialize the discrepancy polynomial according to the higher-order syndromes before each round of the nested KES starts [60]. However, such re-initialization requires many multiplier-adder trees and has long data path.

Algorithm E: Nested KES Algorithm

input: $\Lambda^{(u)}(x)$, $B^{(u)}(x)$, $\hat{\Delta}^{(u)}(x)$, $\hat{\Theta}^{(u)}(x)$, $\gamma^{(u)}$, $k^{(u)}$ from previous KES; S_i $(u \le i < w)$; $S_w = 0$

initialization:

$\hat{\Delta}^{(u)}(x) = \hat{\Delta}^{(u)}(x) + S_u \Lambda^{(u)}(x)$

$\hat{\Theta}^{(u)}(x) = \hat{\Theta}^{(u)}(x) + S_u B^{(u)}(x)$

for $r = u$ to $w - 1$

1). $\Lambda^{(r+1)}(x) = \gamma^{(r)} \Lambda^{(r)}(x) + \hat{\Delta}_0^{(r)} x B^{(r)}(x)$

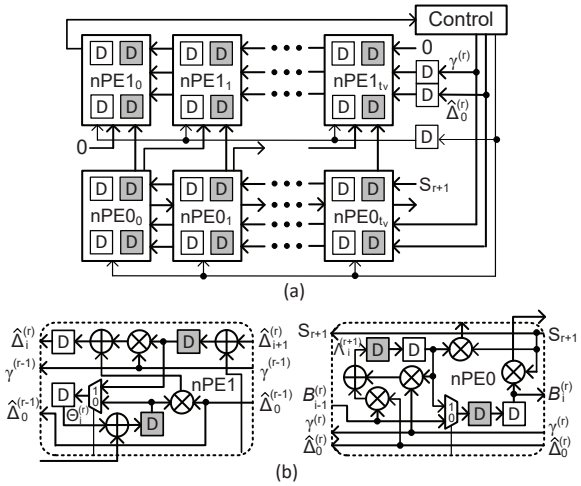

Fig. 11. Architecture for nested KES. (a) overall architecture; (b) nPE architecture

2). $\hat{\Delta}^{(r+1)}(x) = \gamma^{(r)}(\hat{\Delta}^{(r)}(x)/x + S_{r+1}\Lambda^{(r)}(x)) + \hat{\Delta}_0^{(r)}(\hat{\Theta}^{(r)}(x) + S_{r+1}xB^{(r)}(x))$

if $\hat{\Delta}_0^{(r)} \neq 0$ and $k^{(r)} \geq 0$

3). $B^{(r+1)}(x) = \Lambda^{(r)}(x)$
$\hat{\Theta}^{(r+1)}(x) = \hat{\Delta}^{(r)}(x)/x + S_{r+1}\Lambda^{(r)}(x)$
$\gamma^{(r+1)} = \hat{\Delta}_0^{(r)}$; $k^{(r+1)} = -k^{(r)} - 1$

else

4). $B^{(r+1)}(x) = xB^{(r)}(x)$
$\hat{\Theta}^{(r+1)}(x) = \hat{\Theta}^{(r)}(x) + S_{r+1}xB^{(r)}(x)$
$\gamma^{(r+1)} = \gamma^{(r)}$; $k^{(r+1)} = k^{(r)} + 1$

Several designs [30]–[32] have been developed recently to address the challenge of continuing the KES from the results over lower-order syndromes to incorporate higher-order syndromes with short critical path as needed for the nested decoding of GII codes. From (2), the discrepancy coefficient always equals the r-th coefficient of $\Delta^{(r)}(x) = \Lambda^{(r)}(x)S(x)$ if $S(x)$ includes at least the first $r+1$ syndromes. Also it is not necessary to have the correct $\Delta_i^{(r)}$ for $r > i$ available before iteration r of the KES. Accordingly, it was discovered in [30] that the higher-order syndromes can be incorporated one by one into existing KES results to compute the discrepancy coefficients and update the polynomials as in the nested KES algorithm listed in Algorithm E.

Essentially, Algorithm E incorporates higher-order syndromes $S_u, S_{u+1}, \cdots, S_{w-1}$ into existing KES result that covers $S_0, S_1, \cdots, S_{u-1}$. It can be used to continue the KES of a nested decoding round from the riBM result of the sub-codeword decoding or the KES result of the previous nested decoding round. This algorithm is implemented by an architecture consisting of $t_v + 1$ pairs of PEs as shown in Fig. 11(a) [30]. The computations in Line 2 of Algorithm E lead to two multipliers and two adders in the critical path. By applying the slow-down technique [37], every register can be replaced by two registers. Then the registers are moved around by re-timing to achieve the nPE architectures shown in Fig. 11(b), which have one multiplier and one adder in the critical path. It should be noted that the nested decoding of two sub-codewords should be interleaved in such a 2-slow architecture and the initial polynomial coefficients are loaded into the gray and white registers, respectively. If the nested decoding is carried out on one sub-codeword at a time, a dummy zero should be inserted after each valid coefficient through initializing the gray registers to zero.

The number of multipliers in each pair of nPEs equals the number of different product terms in the polynomial updating of the nested KES algorithm. To reduce the number of multipliers, a scaled nested KES algorithm has been developed in [31]. The polynomials are scaled to enable the product term sharing without changing the decoding results. Additionally, the scalar computations are reformulated to keep the same number of multipliers in the critical path. As a result, the number of multipliers in each pair of nPEs is reduced from six as in Fig. 11 to four with one multiplier and two adders/multiplexers in the critical path. In the case of BCH-based GII codes, the odd iterations of the nested KES can be also skipped. However, the odd higher-order syndromes still contribute to the discrepancy coefficients of the even nested KES iterations. Hence, two instead of one higher-order syndromes need to be integrated in each iteration if the odd iterations are skipped. Non-trivial extensions of the nested KES algorithm were proposed in [32] for BCH-based GII codes to skip the odd iterations with one multiplier and three adders/multiplexers in the critical path. Besides, the PEs are simplified by a scaled nested BCH KES algorithm, and the number of PEs is substantially reduced with negligible degradation on the error-correcting performance [32].

VI. Erasure-correcting RS codes for failure recovery

Another important application of RS codes is failure recovery in distributed storage. Erasure-correcting RS codes have been used in traditional failure recovery schemes, such as RAID-6. Besides, there have been much recent developments on LRC and MSR codes [16]. Compared to traditional schemes, they access much fewer symbols for failure recovery, and hence have much shorter latency and less penalty on the network bandwidth. They are essential to the continued scaling of distributed storage. Quite a few LRC and MSR codes are constructed based on RS codes. In [17], [61], data symbols are divided into groups in a hierarchical manner and the parities of extended RS codes are split accordingly to protect individual groups. The GII codes are natural LRC codes since up to $2t_0$ failures can be corrected within a sub-codeword without involving the others. Besides, the locality is further improved by modifying the nesting scheme [15], [19]. The coupled-layered (Clay) codes [18] couple the symbols from multiple RS codewords, and reversible transformations are applied on the coupled symbols before and after RS en/decoding. The major components in the en/decoders of these LRC and MSR codes are RS encoders and erasure decoders.

Conventional RS error-only decoding algorithms introduced in the previous sections can be simplified for erasure decoding since erasures are errors whose locations are known and hence the error locator polynomial can be directly computed.

Nevertheless, more significant complexity reduction is made possible by exploiting the property that the erasure correction capability needed for practical distributed storage systems is typically small, such as 3 or 4. MDS codes are preferred for failure recovery since they require minimum redundancy. Instead of using a generator polynomial, it was proposed in [17] to construct a $(k+e,k)$ e-erasure-correcting MDS code for failure recovery using a parity check matrix

$$H=[V|I]=\begin{bmatrix}1 & 1 & \cdots & 1 & 1 & 0 & \cdots & 0\\ \alpha^0 & \alpha^1 & \cdots & \alpha^{k-1} & 0 & 1 & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & 0\\ \alpha^0 & \alpha^{(e-1)}\cdots & & \alpha^{(e-1)(k-1)} & 0 & 0 & \cdots & 1\end{bmatrix},$$

where V is a Vandermonde matrix of dimension $e\times k$. In order for a code whose H matrix is in such a format to be MDS, every square sub-matrix of V needs to be non-singular. For general k, a Vandermonde matrix is not typically used as the V matrix since this requirement is only satisfied for $e=2$ and 3, in which case the codes are referred to as doubly- and triply-extended, respectively, RS codes in the literature. However, even if $e=4$ or larger, every square sub-matrix of the Vandermonde V matrix is non-singular if k does not exceed a certain limit. Moreover, by exploiting the variation on finite field element representation, the maximum allowed k is increased [34] and is sufficient for many applications.

Using a Vandermonde V matrix leads to great hardware complexity reduction in the en/decoder compared to using a Cauchy matrix, although the latter allows any $k<2^q-e$ for a code over $GF(2^q)$ and hence is conventionally used for MDS code construction. Due to the special structure of the Vandermonde matrix, the parities are computed by simple feedback loops with constant multipliers similar to that for syndrome computation in Fig. 3. Using the H matrix, the erasure magnitudes can be computed by equation solving. The regularity in the Vandermonde matrix helps to cancel out many terms in the equation derivation and leads to simple formulas for erasure calculation. For example, for $e=4$, the erasures, d_x,d_y,d_z,d_w, are computed as [17].

$$\begin{cases} d_z=\frac{\alpha^{w+x+y}q_0+(\alpha^{w+x}+\alpha^{w+y}+\alpha^{x+y})q_1+(\alpha^w+\alpha^x+\alpha^y)q_2+q_3}{(\alpha^w+\alpha^z)(\alpha^x+\alpha^z)(\alpha^y+\alpha^z)}\\ d_y=\frac{\alpha^{w+x}q_0+(\alpha^w+\alpha^x)q_1+q_2+(\alpha^w+\alpha^z)(\alpha^x+\alpha^z)d_z}{(\alpha^w+\alpha^y)(\alpha^x+\alpha^y)}\\ d_x=\frac{\alpha^w q_0+q_1+(\alpha^w+\alpha^y)d_y+(\alpha^w+\alpha^z)d_z}{(\alpha^w+\alpha^x)}\\ d_w=q_0+d_x+d_y+d_z. \end{cases} \tag{7}$$

In the above equations, $\alpha^w,\alpha^x,\alpha^y,\alpha^z$ are the erasure locators. q_0,q_1,q_2,q_3 are the sums of the non-erased symbols and parities multiplied with the corresponding entries in H. Constructing an RS code using the generator polynomial and combining the syndrome computation, KES, and erasure magnitude computation steps can also generate a set of formulas for calculating the erasures directly. However, each formula is in the same format as the d_z equation in (7), which is much more complex than the other three. The architecture for computing (7) is available in [17].

The coupling and nesting of RS codes involved in the Clay and GII codes involve constant finite field elements. These elements do not have to be in the same finite field, $GF(2^q)$, used for RS code construction. For non-prime q, $GF(2^q)$ is isomorphic to composite field $GF((2^m)^n)$ with $q=mn$. Elements from subfield $GF(2^m)$ can be often used to satisfy the constraints for coupling and nesting and the associate computations are greatly simplified [30], [33]. Additionally, there are many different ways to construct fields of a certain order. For examples, $GF(2^8)$ and its composite fields can be constructed in 1050 different ways [62]. The representations of constant field elements and the complexities of the corresponding computations change with the field construction. Hence, the variation on finite field construction can be also exploited to reduce the hardware complexity.

VII. Conclusions

This paper discusses the VLSI architectures for implementing classic RS/BCH codes as well as new codes built by nesting/coupling RS/BCH codes that are potential key enablers of many next-generation technologies. In particular, extensive discussions have been given to the KES step for classic RS/BCH decoding. The most significant KES architectures and fundamental mathematical reformulations leading to the architectures are explained in detail. Besides, this paper introduces the new challenges brought by nesting and coupling RS/BCH codes and the most recent advancements on the corresponding decoder architecture design. RS/BCH codes are continuously finding their ways in new code constructions and applications, as evident in the GII and Clay codes. Although the decoder architecture design for different settings faces various challenges and may demand alternative optimizations and reformulations, many mathematical properties and common techniques can be shared. Hope this paper will facilitate better understanding of classic RS/BCH decoder architectures and help with the architecture design for new codes and applications.

References

[1] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journ. of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

[2] A. Hocquenghem, "Codes correcteurs d'erreurs," *Chiffres*, vol. 2, no. 2, pp. 147–56, 1959.

[3] R. C. Bose and D. K. Ray-Chaudhuri, "On a class of error correcting binary group codes," *Info. and Control*, vol. 3, no. 1, pp. 68–79, 1960.

[4] R. Gallager, "Low-density parity-check codes," *IRE Trans. on Info. Theory*, vol. 8, no. 1, pp. 21–28, 1962.

[5] P. Elias, "Error-free coding," *Trans. IRE Professional Group on Info. Theory*, vol. 4, no. 4, pp. 29–37, 1954.

[6] J. Justesen, K. J. Larsen, and L. A. Pedersen, "Error correcting coding for OTN," *IEEE Commun. Magazine*, vol. 48, no. 9, pp. 70–75, 2010.

[7] B. P. Smith, A. Farhood, A. Hunt, F. R. Kschischang, and J. Lodge, "Staircase codes: FEC for 100 Gb/s OTN," *Journ. of Lightwave Tech.*, vol. 30, no. 1, pp. 110–117, 2011.

[8] H. D. Pfister, S. K. Emmadi, and K. Narayanan, "Symmetric product codes," in *Info. Theory and Applications Workshop (ITA)*, 2015, pp. 282–290.

[9] K. Lee, H.-G. Kang, J.-I. Park, and H. Lee, "A high-speed low-complexity concatenated BCH decoder architecture for 100 Gb/s optical communications," *Springer Journ. of Signal Processing Syst.*, vol. 66, no. 1, pp. 43–55, 2012.

[10] X. Zhang and Z. Wang, "A low-complexity three-error-correcting BCH decoder for optical transport network," *IEEE Trans. on Circuits and Syst.-II*, vol. 59, no. 10, pp. 663–667, 2012.

[11] X. Zhang and M. O'Sullivan, "Ultra-compressed three-error-correcting BCH decoder," in *IEEE Intl. Symp. on Circuits and Syst.*, 2018, pp. 1–5.

[12] C. Fougstedt and P. Larsson-Edefors, "Energy-efficient high-throughput VLSI architectures for product-like codes," *Journal of Lightwave Technology*, vol. 37, no. 2, pp. 477–485, 2019.

[13] X. Tang and R. Koetter, "A novel method for combining algebraic decoding and iterative processing," in *IEEE Intl. Symp. on Info. Theory*, 2006, pp. 474–478.

[14] Y. Wu, "Generalized integrated interleaved codes," *IEEE Trans. on Info. Theory*, vol. 63, no. 2, pp. 1102–1119, 2016.

[15] X. Zhang, "Generalized three-layer integrated interleaved codes," *IEEE Commun. Letters*, vol. 22, no. 3, pp. 442–445, 2017.

[16] S. Balaji, M. N. Krishnan, M. Vajha, V. Ramkumar, B. Sasidharan, and P. V. Kumar, "Erasure coding for distributed storage: An overview," *Science China Info. Sciences*, vol. 61, no. 10, pp. 100–301, 2018.

[17] X. Zhang, S. Sprouse, and I. Ilani, "A flexible and low-complexity local erasure recovery scheme," *IEEE Commun. Letters*, vol. 20, no. 11, pp. 2129–2132, 2016.

[18] M. Vajha *et. al.*, "Clay codes: Moulding MDS codes to yield an MSR code," in *USENIX Conf. on File and Storage Tech.*, 2018, pp. 139–154.

[19] X. Zhang, "Modified generalized integrated interleaved codes for local erasure recovery," *IEEE Commun. Letters*, vol. 21, no. 6, pp. 1241–1244, 2017.

[20] E. Berlekamp, *Algebraic Coding Theory*. McGraw-Hill, 1968.

[21] Y. Sugiyama, M. Kasahara, S. Hirasawa, and T. Namekawa, "A method for solving key equation for decoding Goppa codes," *Info. and Control*, vol. 27, no. 1, pp. 87–99, 1975.

[22] H. Shao, T. Truong, L. Deutsch, J. Yuen, and I. Reed, "A VLSI design of a pipeline Reed-Solomon decoder," in *IEEE Intl. Conf. on Acoustics, Speech, and Signal Processing*, vol. 10, 1985, pp. 1404–1407.

[23] J. H. Baek and M. H. Sunwoo, "New degree computationless modified Euclid algorithm and architecture for Reed-Solomon decoder," *IEEE Trans. on VLSI Syst.*, vol. 14, no. 8, pp. 915–920, 2006.

[24] J. Baek and M. H. Sunwoo, "Simplified degree computationless modified Euclid's algorithm and its architecture," in *IEEE Intl. Symp. on Circuits and Syst.*, 2007, pp. 905–908.

[25] I. S. Reed, M.-T. Shih, and T. K. Truong, "VLSI design of inverse-free Berlekamp-Massey algorithm," *IEE Proceedings Inst. Elect. Eng.-E Computers and Digital Techniques*, vol. 138, no. 5, pp. 295–298, 1991.

[26] D. V. Sarwate and N. R. Shanbhag, "High-speed architectures for Reed-Solomon decoders," *IEEE Trans. on VLSI Syst.*, vol. 9, no. 5, pp. 641–655, 2001.

[27] Y. Wu, "New scalable decoder architectures for Reed-Solomon codes," *IEEE Trans. on Commun.*, vol. 63, no. 8, pp. 2741–2761, 2015.

[28] W. Liu, J. Rho, and W. Sung, "Low-power high-throughput BCH error correction VLSI design for multi-level cell NAND flash memories," in *IEEE Workshop on Signal Processing Syst.*, 2006, pp. 303–308.

[29] Y. Wu, "Binary BCH decoders," *US patent, No. 8,132,08*, 2012.

[30] X. Zhang and Z. Xie, "Efficient architectures for generalized integrated interleaved decoder," *IEEE Trans. on Circuits and Syst.-I*, vol. 66, no. 10, pp. 4018–4031, 2019.

[31] Z. Xie and X. Zhang, "Scaled nested key equation solver for generalized integrated interleaved decoder," *IEEE Trans. on Circuits and Syst.-II*, 2020.

[32] ——, "Reduced-complexity key equation solvers for generalized integrated interleaved BCH decoders," *IEEE Trans. on Circuits and Syst.-I*, 2020.

[33] X. Zhang and Z. Xie, "Efficient VLSI architectures for coupled-layered regenerating codes," *IEEE Trans. on Circuits and Syst.-II*, 2019.

[34] ——, "Relaxing the constraints on locally recoverable erasure codes by finite field element variation," *IEEE Commun. Letters*, vol. 23, no. 10, pp. 1680–1683, 2019.

[35] G. Forney, "On decoding BCH codes," *IEEE Trans. on Info. Theory*, vol. 11, no. 4, pp. 549–557, 1965.

[36] S. B. Wicker, *Error Control Systems for Digital Communication and Storage*. Prentice Hall, 1995.

[37] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. John Wiley & Sons, 2007.

[38] T.-B. Pei and C. Zukowski, "High-speed parallel CRC circuits in VLSI," *IEEE Trans. on Commun.*, vol. 40, no. 4, pp. 653–657, 1992.

[39] J. Jung, H. Yoo, Y. Lee, and I.-C. Park, "Efficient parallel architecture for linear feedback shift registers," *IEEE Trans. on Circuits and Syst.-II*, vol. 62, no. 11, pp. 1068–1072, 2015.

[40] M. Ayinala and K. K. Parhi, "High-speed parallel architectures for linear feedback shift registers," *IEEE Trans. on Signal Processing*, vol. 59, no. 9, pp. 4459–4469, 2011.

[41] X. Zhang, *VLSI Architectures for Modern Error-Correcting Codes*. CRC Press, 2017.

[42] K. Seth, K. Viswajith, S. Srinivasan, and V. Kamakoti, "Ultra folded high-speed architectures for Reed-Solomon decoders," in *IEEE Intl. Conf. on VLSI Design*, 2006, pp. 4–pp.

[43] Y. Chen and K. K. Parhi, "Small area parallel Chien search architectures for long BCH codes," *IEEE Trans. on VLSI Syst.*, vol. 12, no. 5, pp. 545–549, 2004.

[44] Y. Lee, H. Yoo, and I.-C. Park, "Low-complexity parallel Chien search structure using two-dimensional optimization," *IEEE Trans. on Circuits and Syst.-II*, vol. 58, no. 8, pp. 522–526, 2011.

[45] J. Cho and W. Sung, "Strength-reduced parallel Chien search architecture for strong BCH codes," *IEEE Trans. on Circuits and Syst.-II*, vol. 55, no. 5, pp. 427–431, 2008.

[46] H. Yoo, Y. Lee, and I.-C. Park, "Low-power parallel Chien search architecture using a two-step approach," *IEEE Trans. on Circuits and Syst.-II*, vol. 63, no. 3, pp. 269–273, 2015.

[47] S.-Y. Wong, C. Chen, and Q. J. Wu, "Low power Chien search for BCH decoder using RT-level power management," *IEEE Trans. on VLSI Syst.*, vol. 19, no. 2, pp. 338–341, 2009.

[48] X. Zhang, I. Dror, and S. Alterman, "Low-power partial-parallel Chien search architecture with polynomial degree reduction," in *IEEE Intl Symp. on Circuits and Syst.*, 2016, pp. 2459–2462.

[49] K. K. Parhi, "Eliminating the fanout bottleneck in parallel long BCH encoders," *IEEE Trans. on Circuits and Syst.-I*, vol. 51, no. 3, pp. 512–516, 2004.

[50] X. Zhang and K. K. Parhi, "High-speed architectures for parallel long BCH encoders," *IEEE Trans. on VLSI Syst.*, vol. 13, no. 7, pp. 872–877, 2005.

[51] Y. J. Tang and X. Zhang, "Low-complexity architectures for parallel long BCH encoders," in *IEEE Workshop on Signal Processing Syst.*, 2020, p. submitted.

[52] J. H. Derby, "High-speed CRC computation using state-space transformations," in *IEEE Global Telecommunications Conf.*, 2001, pp. 166–170.

[53] G. Hu, J. Sha, and Z. Wang, "High-speed parallel LFSR architectures based on improved state-space transformations," *IEEE Trans. on VLSI Syst.*, vol. 25, no. 3, pp. 1159–1163, 2016.

[54] X. Zhang, "A low-power parallel architecture for linear feedback shift registers," *IEEE Trans. on Circuits and Syst.-II*, vol. 66, no. 3, pp. 412–416, 2018.

[55] X. Zhang and Y. J. Tang, "Reducing parallel linear feedback shift register complexity through input tap modification," in *IEEE Intl. Symp. on Circuits and Syst.*, 2019, pp. 1–5.

[56] X. Zhang, "High-speed and low-complexity parallel long BCH encoder," in *IEEE Intl. Symp. on Circuits and Syst.*, 2020, pp. 1–5.

[57] H. Okano and H. Imai, "A construction method of high-speed decoders using ROM's for Bose-Chaudhuri-Hocquenghem and Reed-Solomon codes," *IEEE Trans. on Computers*, no. 10, pp. 1165–1171, 1987.

[58] X. Zhang, "Systematic encoder of generalized three-layer integrated interleaved codes," in *IEEE Intl. Conf. on Commun.*, 2019, pp. 1–6.

[59] W. Li, J. Tian, J. Lin, and Z. Wang, "Modified GII-BCH codes for low-complexity and low-latency encoders," *IEEE Commun. Letters*, vol. 23, no. 5, pp. 785–788, 2019.

[60] W. Li, J. Lin, and Z. Wang, "A 124-Gb/s decoder for generalized integrated interleaved codes," *IEEE Trans. on Circuits and Syst.-I*, vol. 66, no. 8, pp. 3174–3187, 2019.

[61] C. Huang, M. Chen, and J. Li, "Pyramid codes: flexible schemes to trade space for access efficiency in reliable data storage systems," *ACM Trans. on Storage*, vol. 9, no. 1, pp. 1–28, 2013.

[62] X. Zhang and Y. Lao, "On the construction of composite finite fields for hardware obfuscation," *IEEE Trans. on Computers*, vol. 68, no. 9, pp. 1353–1364, 2019.

PLACE PHOTO HERE

Xinmiao Zhang received her Ph.D. degree in Electrical Engineering from the University of Minnesota. She joined The Ohio State University as an Associate Professor in 2017. Prior to that, she was a Timothy E. and Allison L. Schroeder Associate Professor at Case Western Reserve University. Between her academic positions, she was a Senior Technologist at Western Digital/SanDisk Corporation. Dr. Zhang's research spans the areas of VLSI architecture design, digital storage and communications, security, and signal processing.

Dr. Zhang received an NSF CAREER Award in January 2009. She is also the recipient of the Best Paper Award at 2004 ACM Great Lakes Symposium on VLSI and 2016 International SanDisk Technology Conference. She authored the book "VLSI Architectures for Modern Error-Correcting Codes" (CRC Press, 2015), and co-edited "Wireless Security and Cryptography: Specifications and Implementations" (CRC Press, 2007). She was elected to serve on the Board of Governors of the IEEE Circuits and Systems Society for the 2019-2021 term. She is also a Vice-Chair of the Data Storage Technical Committee (2017-2020), and a member of the CASCOM and VSA technical committees and DISPS technical committee advisory board of IEEE. She served on the technical program and organization committees of many conferences, including ISCAS, SiPS, ICC, GLOBECOM, GlobalSIP, and GLSVLSI. She has been an associate editor for the IEEE Transactions on Circuits and Systems-I 2010-2019 and an associate editor of the IEEE Open Journal of Circuits and Systems since 2019.